Санкт–Петербургский государственный университет

Группа 24.М71-мм

КИСЕЛЕВ Владимир Александрович

Отчет по учебной практике
в форме «Производственное задание»

«Разработка высокопроизводительного MAVLink слоя маршрутизации для стимуляторы роя роботов»

Научный руководитель:
доктор физико-математических наук,
профессор кафедры системного программирования,
Граничин Олег Николаевич

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1 ЦЕЛЬ И ЗАДАЧИ РАБОТЫ 5

2 ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ 6

2.1 Протокол MAVLink и роль промежуточного слоя маршрутизации 6

2.2 Python MAVProxy 6

2.2.1 Ограничения Python MAVProxy в роевой топологии 7

3 ТРЕБОВАНИЯ К СЛОЮ МАРШРУТИЗАЦИИ 8

3.1 Функциональные требования 8

3.2 Нефункциональные требования 8

4 АНАЛИЗ ИСХОДНОЙ PYTHON-BASED ТОПОЛОГИИ 9

4.1 Схема взаимодействия 9

4.1.1 Фактическая схема взаимодействия 9

4.2 Критерий готовности симуляции 9

5 АРХИТЕКТУРА GO-MAVPROXY 11

5.1 Общая структура решения 11

5.2 Основные модули 11

5.3 Поток данных 11

5.4 Изоляция сессий 12

6 ДОСТИЖЕНИЕ ПОВЕДЕНЧЕСКОЙ СОВМЕСТИМОСТИ С PYTHON MAVPROXY 13

6.1 Постановка задачи 13

6.2 Ключевые несовместимости 13

6.2.1 Восстановленная совместимость 13

7 ЛОГИРОВАНИЕ И ДИАГНОСТИЧЕСКИЕ ВОЗМОЖНОСТИ 14

8 АРХИТЕКТУРА ОДНОГО ПРОЦЕССА С НЕСКОЛЬКИМИ ИЗОЛИРОВАННЫМИ СЕССИЯМИ ... 15
8.1 Мотивация ... 15
8.2 Архитектурная идея ... 15
8.3 Преимущества режима ... 15

9 МЕТОДИКА ЭКСПЕРИМЕНТАЛЬНОЙ ОЦЕНКИ ... 16
9.1 Цель экспериментов ... 16
9.2 Условия сравнения ... 16
9.2.1 Метрики ... 16
9.2.2 Масштабы экспериментов ... 17

10 РЕЗУЛЬТАТЫ НАГРУЗОЧНЫХ ТЕСТОВ ... 18
10.1 Сводные таблицы ... 18
10.2 Интерпретация результатов ... 18

11 ВЫВОДЫ ... 20

12 ЗАКЛЮЧЕНИЕ И ДАЛЬНЕЙШАЯ РАБОТА ... 21

ПРИЛОЖЕНИЯ ... 22
Приложение А. Примеры команд запуска ... 22
Приложение Б. Конфигурация go-mavproxy ... 23
Приложение В. Сводная таблица benchmark-результатов ... 24

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... 25

ВВЕДЕНИЕ

Разработка алгоритмов управления роем беспилотных летательных аппаратов требует наличия программной среды, в которой такие алгоритмы могут исследоваться безопасно, воспроизводимо и в условиях, максимально приближённых к реальномым условиям. В предыдущих семестрах была сформирована архитектурная основа симулятора роя, а также реализован высокопроизводительный golang-simulator, позволивший повысить масштабируемость моделирования.

После ускорения основного симуляционного ядра стало очевидно, что следующим ограничивающим звеном системы является промежуточный слой маршрутизации, обеспечивающий взаимодействие между экземплярами ArduPilot SITL и симулятором. В исходной топологии каждому экземпляру SITL соответствовал отдельный процесс Python MAVProxy, через который проходили как телеметрические данные, так и управляющие команды. По мере увеличения числа роботов именно этот слой начал вносить значительный вклад в суммарное потребление памяти и процессорного времени.

Следовательно, дальнейшее развитие симулятора требовало не только оптимизации его вычислительного ядра, но и переосмысления промежуточного программного модуля, расположенного между SITL и golang-simulator.

1 ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Целью работы является разработка, интеграция и экспериментальная оценка go-mavproxy - специализированного слоя маршрутизации на языке Go, предназначенного для замены Python MAVProxy.

Для достижения поставленной цели были сформулированы следующие задачи:

1. Проанализировать роль Python MAVProxy в архитектуре симулятора и формализовать фактическую схему взаимодействия между SITL, промежуточным слоем маршрутизации и golang-simulator.
2. Определить функциональные и нефункциональные требования к новому слою маршрутизации.
3. Разработать архитектуру go-mavproxy с учётом требований совместимости, изоляции сессий и наблюдаемости.
4. Достичь поведенческой совместимости с Python MAVProxy по критическим сценариям, используемым симулятором.
5. Реализовать режим одного процесса с несколькими изолированными сессиями для сокращения накладных расходов промежуточного слоя.
6. Провести сравнительные нагрузочные эксперименты и оценить эффективность трёх режимов работы слоя маршрутизации.

2 ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

2.1 Протокол MAVLink и роль промежуточного слоя маршрутизации

В экосистеме ArduPilot протокол MAVLink используется как универсальное средство обмена сообщениями между автопилотом, наземной станцией и внешними сервисами [1]. В рассматриваемом симуляторе каждый виртуальный робот представлен экземпляром ArduPilot SITL, который должен обмениваться телеметрией и управляющими сообщениями с симулятором физики и, при необходимости, с внешней наземной станцией.

Для организации такого взаимодействия применяется промежуточный слой маршрутизации. Он устанавливает транспортные соединения с SITL, ретранслирует телеметрию на внешние выходы и обеспечивает доставку обратных управляющих команд.

2.2 Python MAVProxy

Python MAVProxy является распространённым инструментом в экосистеме ArduPilot [2]. Он поддерживает различные типы транспортов, интерактивную консоль, пересылку трафика, ведение логов телеметрии и интеграцию с внешними GCS. Однако в рассматриваемом пайплайне взаимодействия симулятора используется не весь набор его возможностей, а лишь ограниченное подмножество:

- открытие основного TCP-канала к экземпляру SITL;
- передача телеметрии на UDP-выходы;
- приём управляющих команд от симулятора и их возврат в SITL;
- ведение логов, пригодных для диагностики трафика.

Иными словами, в рамках данной работы Python MAVProxy рассматривается как корректное базовое решение, относительно которого оценивается поведение новой реализации.

2.2.1 Ограничения Python MAVProxy в роевой топологии

При работе с одним экземпляром SITL Python MAVProxy обеспечивает ожидаемое поведение и представляет собой удобное базовое решение. Однако в роевой топологии каждому экземпляру SITL соответствует отдельный процесс MAVProxy. Такая организация приводит к линейному росту числа процессов и, как следствие, к увеличению суммарных накладных расходов на память, планирование и выполнение.

После оптимизации основного Go-симулятора именно слой маршрутизации стал новым ограничивающим фактором системы.

3 ТРЕБОВАНИЯ К СЛОЮ МАРШРУТИЗАЦИИ

3.1 Функциональные требования

Анализ поведения Python MAVProxy в составе симулятора позволил сформулировать следующие функциональные требования к новому слою маршрутизации:

- открытие основного TCP-соединения с каждым экземпляром SITL;
- поддержка обратного канала для передачи команд от симулятора в SITL;
- маршрутизация телеметрии с основного канала на UDP-выходы, используемые golang-simulator и внешними GCS;
- корректная ранняя инициализация канала без потери первых сообщений;
- запрос потоков телеметрии;
- строгая изоляция сессий, исключающая смешение телеметрии и команд разных роботов;
- совместимость с используемым подмножеством CLI- и config-синтаксиса Python MAVProxy.

3.2 Нефункциональные требования

Наряду с функциональной корректностью новый слой маршрутизации должен удовлетворять следующим нефункциональным требованиям:

- уменьшать потребление памяти и CPU по сравнению с Python MAVProxy;
- поддерживать более эффективную организацию выполнения в режиме одного процесса с несколькими изолированными сессиями;
- обеспечивать наблюдаемость через раздельные журналы приложения, журналы сообщений и бинарные файлы .tlog;
- быть пригодным для воспроизводимых сравнительных нагрузочных экспериментов в составе симулятора.

4 АНАЛИЗ ИСХОДНОЙ PYTHON-BASED ТОПОЛОГИИ

4.1 Схема взаимодействия

В исходной топологии каждый экземпляр SITL подключается к отдельному экземпляру Python MAVProxy. Этот модуль маршрутизации выполняет две основные функции. Во-первых, он пересылает MAVLink-телеметрию с основного TCP-канала на UDP-выход, откуда её принимает golang-simulator. Во-вторых, он принимает обратные команды от симулятора и передаёт их в SITL через обратный канал, что делает из него обязательный транспортный узел, через который проходят оба критически важных потока данных: телеметрический и управляющий.

4.1.1 Фактическая схема взаимодействия

Анализ поведения Python MAVProxy, принятого в качестве базового решения, показал, что в проекте от него требуется не полный интерактивный набор возможностей, а ограниченная, но критически важная схема взаимодействия:

1. надёжное открытие основного TCP-канала к SITL;
2. своевременное появление HEARTBEAT и GLOBAL_POSITION_INT;
3. стабильная передача телеметрии на выходы симулятора;
4. доставка команд обратно в SITL;
5. отсутствие смешения данных между сессиями.

4.2 Критерий готовности симуляции

Для корректного запуска сценария требуется, чтобы симулятор получил от каждого робота базовые MAVLink-сообщения:

- HEARTBEAT;
- GLOBAL_POSITION_INT.

После получения этих сообщений golang-simulator фиксирует готовность роя сообщением `All drones are connected and have a position fix`. Отсюда следует, что потеря первого heartbeat, отсутствие запроса необходимых потоков телеметрии или нарушение обратного пути команд делают невозможным выполнение сценария полёта.

5 АРХИТЕКТУРА GO-MAVPROXY

5.1 Общая структура решения

go-mavproxy спроектирован как специализированный модуль маршрутизации для симулятора. Его основная задача состоит не в полном воспроизведении пользовательского интерфейса Python MAVProxy, а в реализации той транспортной схемы и того уровня обмена сообщениями, на которые опирается golang-simulator.

5.2 Основные модули

Структура программы включает следующие ключевые компоненты:

- `cmd/go-mavproxy` — точка входа, парсинг аргументов и запуск сессий выполнения;
- `pkg/config` — загрузка конфигурации и объединение семантики командной строки и конфигурационного файла;
- `pkg/router` — маршрутизация сообщений между основным каналом, UDP-выходами и обратным каналом `sitl`;
- `pkg/logger` — ведение журналов приложения и журналов сообщений;
- `pkg/tlog` — запись бинарного журнала телеметрии;
- `gomavlib` — низкоуровневая библиотека для работы с транспортным уровнем и MAVLink-кадрами.

5.3 Поток данных

Телеметрия от SITL поступает в модуль маршрутизации по основному TCP-каналу, после чего соответствующий маршрутизатор пересылает её на заданные UDP-выходы. Управляющие команды от golang-simulator, в свою очередь, принимаются через выходные каналы и передаются обратно в SITL через обратный канал. Такая организация делает двустороннюю

структуру обмена явной и позволяет контролировать все точки взаимодействия.

5.4 Изоляция сессий

Одним из ключевых архитектурных требований является строгая изоляция сессий. Для каждого SITL необходимо собственное состояние маршрутизации, собственные выходные каналы и собственный обратный канал. Это требование сохраняется и при переходе к режиму одного процесса с несколькими изолированными сессиями, где логическая изоляция должна соблюдаться даже внутри одного процесса операционной системы.

6 ДОСТИЖЕНИЕ ПОВЕДЕНЧЕСКОЙ СОВМЕСТИМОСТИ С PYTHON MAVPROXY

6.1 Постановка задачи

В рамках данной работы поведенческая совместимость понимается не как полное воспроизведение всех пользовательских возможностей Python MAVProxy, а как достижение эквивалентности по тем функциям, которые фактически используются в основном сценарии использования симулятора. Такой подход позволяет сосредоточиться на инженерно значимой части поведения и обеспечивает корректную основу для последующего сравнительного анализа производительности.

6.2 Ключевые несовместимости

В ходе анализа и доведения поведения go-mavproxy до уровня базового решения были выявлены следующие критические несовместимости:

- риск потери первого HEARTBEAT на этапе запуска маршрутизации;
- неполная совместимость поведения streamrate;
- отсутствие поддержки встроенных комментариев в конфигурации;
- различия в поведении при передачи данных через UDP-выходы;
- отсутствие разделения между журналом приложения и журналом телеметрии.

6.2.1 Восстановленная совместимость

После устранения указанных различий go-mavproxy стал воспроизводить критически важное поведение Python MAVProxy, принятого в качестве базового решения.

7 ЛОГИРОВАНИЕ И ДИАГНОСТИЧЕСКИЕ ВОЗМОЖНОСТИ

Python MAVProxy используется в проекте не только как средство маршрутизации, но и как удобный диагностический инструмент. Поэтому в go-mavproxy была спроектирована отдельная подсистема логирования, включающая несколько типов артефактов:

- журнал приложения — события жизненного цикла, открытие каналов, предупреждения и ошибки;
- журнал сообщений — человекочитаемое представление проходящего MAVLink-трафика с указанием направления, канала и типа сообщения;
- бинарный журнал телеметрии — файл, предназначенный для последующего анализа необработанных кадров.

Разделение журналов по назначению повышает наблюдаемость системы, облегчает локализацию проблем на транспортном уровне и делает сравнительные нагрузочные эксперименты более корректными, поскольку накладные расходы на журналирование могут контролироваться отдельно.

8 АРХИТЕКТУРА ОДНОГО ПРОЦЕССА С НЕСКОЛЬКИМИ ИЗОЛИРОВАННЫМИ СЕССИЯМИ

8.1 Мотивация

Даже после достижения поведенческой совместимости схема «один процесс на одного робота» сохраняет принципиальное ограничение: каждому роботу соответствует отдельный процесс маршрутизации. Это приводит к дублированию накладных расходов среды выполнения, росту суммарного потребления памяти и увеличению нагрузки на планировщик операционной системы.

8.2 Архитектурная идея

Для устранения этого ограничения был реализован режим одного процесса с несколькими изолированными сессиями. В нём один процесс обслуживает несколько runtime сессий, причём каждая сессия сохраняет:

- собственное основное TCP-соединение;
- собственные выходные каналы;
- собственный обратный канал;
- собственное состояние маршрутизации;
- собственные логические артефакты.

8.3 Преимущества режима

Такая организация обеспечивает одновременно два результата:

- сохраняется логическая изоляция между роботами;
- устраняется дублирование процессов, что снижает peak RSS и peak aggregate CPU.

Следовательно, выигрыш указанного режима обусловлен не только выбором языка Go, но и более эффективной архитектурной организацией промежуточного слоя маршрутизации.

9 МЕТОДИКА ЭКСПЕРИМЕНТАЛЬНОЙ ОЦЕНКИ

9.1 Цель экспериментов

Экспериментальная часть работы направлена на сравнение трёх режимов слоя маршрутизации в одинаковых условиях взаимодействия симулятора:

1. Python MAVProxy в режиме одного процесса на робота;
2. go-mavproxy в режиме одного процесса на робота;
3. go-mavproxy в режиме одного процесса с несколькими изолированными сессиями.

9.2 Условия сравнения

Во всех случаях использовались одинаковые сценарии запуска, одинаковые конфигурации роботов и единый критерий готовности. Это необходимо для того, чтобы наблюдаемые различия объяснялись именно организацией слоя маршрутизации, а не побочными факторами окружения.

9.2.1 Метрики

В ходе экспериментов фиксировались следующие показатели:

- **Readiness time** — время до появления сообщения `All drones are connected and have a position fix`;
- **Peak RSS** — пиковое суммарное потребление резидентной памяти процессами маршрутизации;
- **Peak aggregate CPU** — пиковая суммарная загрузка CPU процессами маршрутизации;
- **Success/failure** — успешность прохождения сценария запуска.

9.2.2 Масштабы экспериментов

Нагрузочные тесты проводились для 4, 8, 16 и 32 экземпляров SITL. Такой набор масштабов позволяет проследить не только локальные различия между режимами, но и характер роста накладных расходов по мере увеличения числа роботов.

10 РЕЗУЛЬТАТЫ НАГРУЗОЧНЫХ ТЕСТОВ

10.1 Сводные таблицы

Результаты сравнительных экспериментов приведены в таблицах 1–4.

Режим	Readiness (ms)	Peak RSS (KB)	Peak CPU (%)	Результат
python_per_instance	18000	360808	95.00	timeout
go_per_instance	11000	71372	158.80	success
go_single_process	12000	19256	30.90	success

Таблица 1 — Результаты для 4 SITL

Режим	Readiness (ms)	Peak RSS (KB)	Peak CPU (%)	Результат
python_per_instance	28000	724996	314.00	success
go_per_instance	13000	127552	156.90	success
go_single_process	12000	29244	36.20	success

Таблица 2 — Результаты для 8 SITL

Режим	Readiness (ms)	Peak RSS (KB)	Peak CPU (%)	Результат
python_per_instance	31000	1451444	570.50	success
go_per_instance	14000	245784	251.20	success
go_single_process	14000	58156	44.80	success

Таблица 3 — Результаты для 16 SITL

Режим	Readiness (ms)	Peak RSS (KB)	Peak CPU (%)	Результат
python_per_instance	63000	2902012	935.40	success
go_per_instance	18000	479576	387.20	success
go_single_process	18000	89276	49.30	success

Таблица 4 — Результаты для 32 SITL

10.2 Интерпретация результатов

Полученные результаты позволяют сделать три вывода:

1. go-mavproxy воспроизводит критически важное для симулятора поведение, ранее обеспечиваемое Python MAVProxy.
2. Уже в режиме одного процесса на робота реализация на Go демонстрирует существенно меньший объём потребляемой памяти по сравнению с Python-реализацией.
3. Наибольший эффект достигается в режиме одного процесса с несколькими изолированными сессиями, где при сохранении корректного поведения резко снижаются peak RSS и peak aggregate CPU.

Следовательно, основной выигрыш обусловлен не только переписыванием промежуточного слоя на Go, но и переходом от схемы «один процесс на одного робота» к архитектуре одного процесса с несколькими изолированными сессиями.

11 ВЫВОДЫ

В результате выполнения работы были получены следующие основные выводы:

1. Python MAVProxy, являясь корректным базовым решением, при масштабировании становится одним из основных ограничений симулятора по памяти и CPU.
2. go-mavproxy обеспечивает поведенческую совместимость по критическим функциям, необходимым `golang-simulator` для корректного выполнения сценариев.
3. Режим одного процесса с несколькими изолированными сессиями позволяет сохранить изоляцию сессий и одновременно существенно сократить накладные расходы слоя маршрутизации.
4. В наиболее показательном эксперименте для `32 SITL` указанный режим снижает peak RSS с порядка `2.9 GB` до `89 MB`, а peak aggregate CPU — с `935.40%` до `49.30%`.

Направления дальнейшего развития:

- Дальнейшее развитие подсистем журналирования и вспомогательного инструментария;
- Расширение поддержки сценариев взаимодействия с внешними GCS;
- Дополнительные оптимизации gomavlib и транспортного модуля симулятора.

12 ЗАКЛЮЧЕНИЕ И ДАЛЬНЕЙШАЯ РАБОТА

В рамках настоящей работы была успешно решена поставленная цель: разработан, интегрирован и экспериментально оценен go-mavproxy — специализированный слой маршрутизации на языке Go, предназначенный для замены Python MAVProxy.

В ходе работы были выполнены все поставленные задачи:

1. Проанализирована роль Python MAVProxy в архитектуре симулятора.
2. Определены функциональные и нефункциональные требования к новому слою маршрутизации.
3. Спроектирована архитектура go-mavproxy.
4. Достигнута поведенческая совместимость с Python MAVProxy.
5. Реализован режим одного процесса с несколькими изолированными сессиями.
6. Проведено сравнительное тестирование производительности и оценена эффективность трёх режимов работы слоя маршрутизации.

Ссылки на github репозитории, содержащие материалы работы:

1. https://github.com/CroccoRush/swarm-simulator
2. https://github.com/CroccoRush/go-mavproxy
3. https://github.com/CroccoRush/gomavlib

ПРИЛОЖЕНИЯ

Приложение А. Примеры команд запуска

А.1 Запуск симуляции с go-mavproxy

По умолчанию симулятор использует go-mavproxy в режиме per-instance:

```
./run_golang_simulation.sh experiment direct
```

Режим single-process multi-session задаётся переменной окружения:

```
MAVPROXY_IMPL=go MAVPROXY_TOPOLOGY=single-process \
  ./run_golang_simulation.sh experiment direct
```

Для сравнения с Python MAVProxy:

```
MAVPROXY_IMPL=python MAVPROXY_TOPOLOGY=per-instance \
  ./run_golang_simulation.sh experiment direct
```

А.2 Запуск бенчмарков производительности

Скрипт compare_performance.sh сравнивает три режима по метрикам readiness, peak RSS и peak CPU:

```
# Сравнение всех режимов для конфигурации из CONFIG_FILE
./compare_performance.sh

# Указание конфигурации и числа прогонов
CONFIG_FILE=config_4drones.json RUNS=3 ./compare_performance.sh
```

Переменные окружения:

- `CONFIG_FILE` — путь к JSON-конфигурации дронов;
- `RUNS` — число прогонов для каждого режима;
- `RESULT_DIR` — каталог для сохранения результатов.

Приложение Б. Конфигурация go-mavproxy

Б.1 Аргументы командной строки

go-mavproxy поддерживает аргументы, совместимые с MAVProxy:

```
go-mavproxy --master tcp:127.0.0.1:5760 \
  --sitl 127.0.0.1:5761 \
  --out udp:127.0.0.1:14550 \
  --out udp:127.0.0.1:14551
```

- `-master tcp:HOST:PORT` — master TCP-канал к SITL;
- `-sitl HOST:PORT` — backchannel для команд в SITL;
- `-out udp:HOST:PORT` — UDP-выход (симулятор, GCS).

Б.2 Конфигурационный файл

В конфигурационном файле можно задать дополнительные UDP-выходы:

```
# Дополнительный выход для Mission Planner
output add 127.0.0.1:14552
```

Комментарии # обрабатываются при разборе.

Приложение B. Сводная таблица benchmark-результатов

SITL	Режим	Readiness	Peak RSS	Peak CPU
4	python_per_instance	18000 ms	360808 KB	95.00%
	go_per_instance	11000 ms	71372 KB	158.80%
	go_single_process	12000 ms	19256 KB	30.90%
8	python_per_instance	28000 ms	724996 KB	314.00%
	go_per_instance	13000 ms	127552 KB	156.90%
	go_single_process	12000 ms	29244 KB	36.20%
16	python_per_instance	31000 ms	1451444 KB	570.50%
	go_per_instance	14000 ms	245784 KB	251.20%
	go_single_process	14000 ms	58156 KB	44.80%
32	python_per_instance	63000 ms	2902012 KB	935.40%
	go_per_instance	18000 ms	479576 KB	387.20%
	go_single_process	18000 ms	89276 KB	49.30%

Таблица 5 — Сводная таблица результатов нагрузочных тестов

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Team A.D.* SITL Simulator (Software in the Loop). — 2024. — Accessed: 2025-09-21. https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html.
2. *Team M.D.* MAVProxy - MAVLink proxy and command line ground station. — 2024. — Accessed: 2025-09-21. https://ardupilot.org/mavproxy/.